

Ejercicio 8: Matriz de Rastreo (Divide y Vencerás)

Enunciado

Una matriz de rastreo es una matriz cuadrada de $n \times n$ números enteros (sin repeticiones), siendo n una potencia de 2, y en la que se cumple que $a_1 < b_1 < a_2 < b_2 < a_3 < b_3 < a_4 < b_4$ y donde además las cuatro sub-matrices componentes son también matrices de rastreo.

Diseñar un algoritmo de tipo Divide y Vencerás que determine si la matriz dada es o no de rastreo. Analizar su coste en tiempo.

1. Estrategia de Divide y Vencerás

Por la definición del problema, una matriz M de tamaño $n \times n$ es una **Matriz de Rastreo** si y solo si cumple las siguientes tres condiciones:

1. **Caso Base:** Si $n = 1$, una matriz de 1×1 siempre es una matriz de rastreo válida (un único elemento no puede tener repeticiones ni violar desigualdades).
2. **Propiedad de las Esquinas (Paso de Combinación):** Los elementos de las esquinas clave de las cuatro submatrices de tamaño $(n/2) \times (n/2)$ deben cumplir la cadena de desigualdades estrictas:

$$a_1 < b_1 < a_2 < b_2 < a_3 < b_3 < a_4 < b_4$$

Donde identificamos los índices en la matriz M (usando indexación de 1 a n):

- $a_1 = M[1][1]$
 - $b_1 = M[n/2][n/2]$
 - $a_2 = M[1][n/2 + 1]$
 - $b_2 = M[n/2][n]$
 - $a_3 = M[n/2 + 1][1]$
 - $b_3 = M[n][n/2]$
 - $a_4 = M[n/2 + 1][n/2 + 1]$
 - $b_4 = M[n][n]$
3. **Condición Recursiva (Divide y Vencerás):** Las cuatro submatrices componentes de tamaño $(n/2) \times (n/2)$ deben ser, a su vez, matrices de rastreo válidas.

2. Algoritmo en Pseudocódigo

Para poder subdividir la matriz sin realizar copias físicas (lo cual degradaría la eficiencia espacial y temporal), el algoritmo trabaja con coordenadas límites de la submatriz actual:

(fila_inicio, fila_fin) y (col_inicio, col_fin) .

```
// Función recursiva principal
Función EsMatrizRastreo(M, f_ini, f_fin, c_ini, c_fin):
    n = f_fin - f_ini + 1 // Tamaño de la submatriz actual

    // CASO BASE: Matriz de 1x1
    Si (n == 1):
        Retornar Verdadero

    // 1. Identificar las coordenadas divisorias (mitad)
    mitad = n / 2
    f_med = f_ini + mitad - 1
    c_med = c_ini + mitad - 1

    // 2. Extraer los elementos clave de las esquinas (a_i, b_i)
    a1 = M[f_ini][c_ini]
    b1 = M[f_med][c_med]

    a2 = M[f_ini][c_med + 1]
    b2 = M[f_med][c_fin]

    a3 = M[f_med + 1][c_ini]
    b3 = M[f_fin][c_med]

    a4 = M[f_med + 1][c_med + 1]
    b4 = M[f_fin][c_fin]

    // 3. Comprobar la restricción de desigualdad del nivel actual
    Si NO (a1 < b1 y b1 < a2 y a2 < b2 y b2 < a3 y a3 < b3 y b3 < a4 y a4 < b4):
        Retornar Falso

    // 4. DIVIDIR: Comprobar recursivamente las 4 submatrices componentes
    sub1 = EsMatrizRastreo(M, f_ini, f_med, c_ini, c_med) // Sup-Izq
    sub2 = EsMatrizRastreo(M, f_ini, f_med, c_med + 1, c_fin) // Sup-Der
    sub3 = EsMatrizRastreo(M, f_med + 1, f_fin, c_ini, c_med) // Inf-Izq
    sub4 = EsMatrizRastreo(M, f_med + 1, f_fin, c_med + 1, c_fin) // Inf-Der

    // Vencer: Solo es de rastreo si las cuatro submatrices lo son
    Retornar (sub1 y sub2 y sub3 y sub4)

// Función envoltura (Llamada inicial)
Función ValidarRastreo(M, n):
    Si (n == 0 o NO es_potencia_de_2(n)):
        Retornar Falso
    Retornar EsMatrizRastreo(M, 1, n, 1, n)
```

3. Análisis de Coste Temporal

Para analizar la eficiencia temporal, planteamos la ecuación de recurrencia del algoritmo:

- En cada paso no base de tamaño n , el algoritmo realiza una cantidad constante de operaciones $\mathcal{O}(1)$ para calcular los índices intermedios, acceder a los 8 elementos de las esquinas y comprobar las desigualdades.

- Posteriormente, divide el problema realizando **4 llamadas recursivas** sobre problemas de tamaño mitad ($n/2$).

La ecuación de recurrencia es:

$$T(n) = 4T\left(\frac{n}{2}\right) + \mathcal{O}(1)$$

Aplicación del Teorema Máster

El Teorema Máster se define para ecuaciones de la forma $T(n) = aT(n/b) + \Theta(n^d)$:

- $a = 4$ (número de subproblemas)
- $b = 2$ (factor de reducción de tamaño)
- $d = 0$ (ya que el trabajo de combinación/división es constante, $\mathcal{O}(1) = \Theta(n^0)$)

Comparamos los valores:

$$\log_b(a) = \log_2(4) = 2$$

Dado que $\log_b(a) > d$ ($2 > 0$), nos encontramos en el caso donde el coste está dominado por las llamadas recursivas de las hojas del árbol:

$$T(n) \in \Theta(n^{\log_b(a)}) = \Theta(n^2)$$